

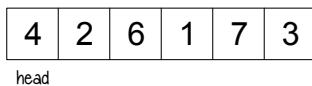
# Cola de valores enteros

Una cola es una estructura de datos de tipo **FIFO** (*First In, First Out*), esencial en programación para gestionar tareas en orden, como el manejo de procesos en sistemas operativos y la gestión de solicitudes en redes.

Implementa las clases indicadas a continuación usando técnicas de **programación orientada a objetos** en Python.

Recuerda que puedes añadir todos aquellos métodos auxiliares que creas necesarios.

## 1. Clase IntegerQueue



Almacena la cola como una **lista** donde el **head** de la cola esté en la posición 0:  
[4, 2, 6, 1, 7, 3]

Implementa, *al menos*, los siguientes **métodos** (*tratando de hacer uso de métodos que vayas implementando*):

```
def __init__(self, *, max_size: int = 10):
```

- Crea los atributos: `items` y `max_size`.
- Ten en cuenta que `items` almacenará los elementos de la cola.

```
def __len__(self) -> str:
```

- Devuelve el número de elementos que hay en la cola `self`.

```
def is_empty(self) -> bool:
```

- Indica si la cola `self` está vacía.

```
def is_full(self) -> bool:
```

- Indica si la cola `self` está llena.

```
def enqueue(self, item: int) -> bool:
```

- Añade `item` a la cola `self`.
- Si la cola está llena habrá que retornar `False`. En otro caso retornar `True`.

```
def dequeue(self) -> int | None:
```

- Extrae (y devuelve) el elemento que está en el **head** de la cola **self**.
- Si la cola está vacía habrá que retornar **None**.

```
def head(self) -> int | None:
```

- Devuelve el elemento que está en el **head** de la cola **self**.
- Si la cola está vacía habrá que retornar **None**.

```
def expand(self, factor: int = 2) -> None:
```

- Expande el tamaño máximo de la cola **self** en el factor indicado.
- Por ejemplo, si el tamaño máximo es 20 y el factor es 3, el tamaño máximo pasa a ser  $20 \times 3 = 60$ .

```
def dump_to_file(self, path: str) -> None:
```

- Vuelca la cola **self** a un fichero con ruta **path**.
- Todos los elementos de la cola estarán en una misma línea separados por comas.
- El primer elemento del fichero coincide con el elemento del **head** de la cola.

```
def load_from_file(cls, path: str) -> IntegerQueue:
```

- Es un **método de clase**
- Construye (y devuelve) una cola desde el fichero con ruta **path**.
- Todos los elementos de la cola estarán en una misma línea separados por comas.
- El primer elemento del fichero coincide con el elemento del **head** de la cola.
- Si la cola se llena al ir añadiendo elementos, habrá que expandir la cola mediante **expand()** con el valor por defecto de **factor**.

```
def __getitem__(self, index: int) -> int:
```

- Devuelve el elemento de la cola **self** que ocupa la posición (*índice*) **index**.

```
def __setitem__(self, index: int, item: int) -> None:
```

- Asigna el elemento **item** en la posición (*índice*) **index** de la cola **self**.

```
def __add__(self, other: IntegerQueue) -> IntegerQueue:
```

- Suma las colas **self** y **other**.
- La segunda cola **other** va “detrás” de la primera **self**.
- El tamaño máximo de la cola resultante es la suma de los tamaños máximos de cada cola.
- Devuelve la cola resultante.

```
def __iter__(self) -> IntegerQueueIterator:
```

- Devuelve un objeto iterador de tipo `IntegerQueueIterator`.

## 2. Clase `IntegerQueueIterator`

Representa un iterador para la clase `IntegerQueue`.

Implementa, *al menos*, los siguientes **métodos**:

```
def __init__(self, stack: IntegerStack) -> None:  
    ...
```

```
def __next__(self) -> int:  
    ...
```